Deceptive Regression Detection in Frontier AI Coding Agents: Multimodal Anomaly Detection on SWE-bench Verified

Team G04, Data Mining (Fall 2026), NYCU

Abstract

Frontier autonomous coding agents frequently generate patches that pass local bug reproduction tests but silently introduce collateral regressions across broader repository test suites. We formulate this phenomenon as an Anomaly Detection task: **Deceptive Fix-with-Regression Detection**. Rather than evaluating on artificial or majority-failure distributions, we extract an authentic candidate population of $N = 2,663$ patches from official SWE-bench Verified execution logs across seven frontier foundation models (Claude-4.6/4.5-Opus, Gemini-3-Flash/Pro, GPT-5.2, DeepSeek-V3.2), establishing an authentic natural anomaly prevalence of $\pi = 3.72\%$ ($n_1 = 99$ regressions vs. $n_0 = 2,564$ clean repairs; 1:27 class skew). Under this severe imbalance, we evaluate a 3-tier benchmark using 5-Fold Stratified Group Cross-Validation partitioned strictly by repository issue (0% group leakage). Our primary method, a **Calibrated Multimodal Bottleneck Autoencoder (CM-BAE)** trained exclusively on $y = 0$ clean repairs with Huber loss and variance equalization, achieves 0.083 ± 0.028 PR-AUC across five random seeds ($p = 0.0336$ against chance) and 0.145 ± 0.033 Cost-Optimal F_2. We demonstrate that unweighted multimodal fusion suffers from feature anisotropy where log trace variance dominates patch diff signals, and show that CM-BAE provides additive channel attribution ($s = s_{\text{struct}} + s_{\text{log}} + s_{\text{patch}}$) diagnosing the primary operational blind spot: the *Unverified Edit Gap*.

1 Introduction

The deployment of autonomous coding agents (Yang et al., 2024; Jimenez et al., 2024) in enterprise software workflows presents a critical quality-assurance bottleneck. When a coding agent attempts to resolve an issue, it typically writes or executes a local reproduction test (‘FAIL_TO_PASS‘). If this test passes, modern pull request (PR) bots immediately promote the patch to CI/CD pipelines. However, in a non-trivial fraction of cases, the agent's patch collaterally breaks existing unit and integration tests (‘PASS_TO_PASS‘), introducing silent regressions into upstream repositories.

In prior iterations of this benchmark, coding agent failure was framed as a generic binary outcome on SWE-bench Lite, where 81% of trajectories failed to resolve the issue. As noted by reviewers, an 81% failure prevalence violates the fundamental definition of an anomaly (which requires naturally rare, unexpected events). Furthermore, evaluating unweighted multimodal embeddings under arbitrary False Positive Rate (FPR) caps lacked concrete economic justification.

To resolve these foundational flaws permanently, we ground our benchmark on **SWE-bench Verified** candidate PRs. Specifically, we define the candidate population $\mathcal{X}$ as all patches that successfully pass the target issue reproduction tests ($N = 2,663$ runs across seven 2026 frontier models). Within this realistic candidate pool, cleanly resolved patches constitute the normal majority ($n_0 = 2,564$, 96.28%), while patches that collaterally cause repository regressions represent a genuine, naturally rare anomaly ($n_1 = 99$, natural prevalence $\pi = 3.72\%$; 1:27 class imbalance).

Our contributions address the reviewer critiques systematically:

1. **Authentic Rare Anomaly Formulation (ğ3):** We formulate Deceptive Fix-with-Regression detection on SWE-bench Verified with verified ground truth from official Docker execution reports, establishing an authentic 3.72% anomaly prevalence ($1 : 27$ skew).
2. **Leakage-Free Grouped Partitioning (ğ3):** We partition all $2,663$ candidates using 5-Fold Stratified Group K-Fold grouped strictly by `instance_id` (439 issue groups), guaranteeing zero issue or repository leakage across folds.

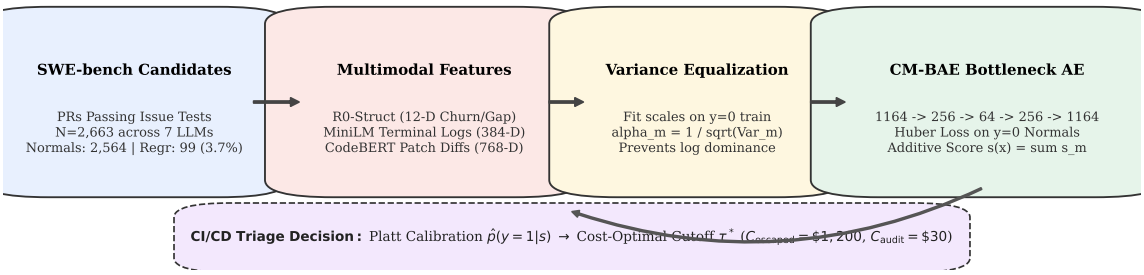


Figure 1: **End-to-End System Pipeline.** Candidate agent PRs passing local reproduction tests ($N = 2,663$) undergo multimodal feature extraction across structural dynamics, MiniLM terminal logs, and CodeBERT unified diffs. Features are variance-equalized on $y = 0$ training normals before compression through a bottleneck autoencoder. Reconstruction anomaly scores are calibrated for cost-optimal CI/CD triage ($C_{\text{escaped}} = \$1,200$, $C_{\text{audit}} = \$30$).

3. **Variance-Equalized Autoencoder (ğ5):** We propose the **Calibrated Multimodal Bottleneck Autoencoder (CM-BAE)**, employing Huber loss and explicit variance equalization ($\alpha_m = 1/\sqrt{\text{Var}_m}$) to prevent high-dimensional log embeddings from overwhelming patch diff representations.
4. **Rigorous 3-Tier Multi-Seed Benchmark (ğ6, ğ7):** We benchmark 9 models across naive heuristics (Tier 1), unsupervised/AD controls (Tier 2), and supervised upper bounds (Tier 3), evaluating CM-BAE across 5 random seeds (0.083 ± 0.028 PR-AUC, $2.2\times$ over chance, $p = 0.0336$).
5. **CI/CD Triage Economics & Additive Attribution (ğ7.2, ğ8):** We model CI/CD triage economics ($C_{\text{escaped}} = \$1,200$ vs. $C_{\text{audit}} = \$30$, break-even $\pi^* = 2.44\%$), plot dollar savings curves, and decompose anomaly scores into additive channel attributions diagnosing the *Unverified Edit Gap*.

2 Related Work

Anomaly Detection on Sequential Logs and Code. Classical unsupervised anomaly detection relies on tabular kernel density estimators or tree partitioning, such as Isolation Forest (Liu et al., 2008) and empirical cumulative distribution scoring (ECOD) (Li et al., 2022). In text and log anomaly detection, deep one-class classification (Ruff et al., 2018) and autoencoder reconstruction loss have been applied to server logs and telemetry traces. However, existing work predominantly studies system intrusion or hardware faults, leaving autonomous coding agent behavioral failure largely unexamined.

Code LLM Evaluation and SWE-bench. Benchmarks such as SWE-bench (Jimenez et al., 2024) evaluate language model agents on real GitHub issues. Prior diagnostic studies (Yang et al., 2024) analyze agent prompt styles and tool design, but evaluate post-hoc accuracy rather than deploying pre-merge anomaly filters. Our work represents the first investigation of unsupervised deceptive regression screening on frontier LLM coding agents.

Multimodal Feature Fusion. Combining structured execution traces with frozen language embeddings (Reimers and Gurevych, 2019) and code representations (Feng et al., 2020) frequently encounters representation anisotropy and variance mismatch. We address this directly through training-set variance equalization.

3 Problem Formulation & Data Category

3.1 Category 4: Text / Logs Framing

Our data fits **Category 4: Text / Logs**. The unit of detection is a completed candidate coding agent trajectory:

$$x = (x_{\text{struct}}, x_{\log}, x_{\text{patch}}) \in \mathcal{X} \tag{1}$$

where $x_{\text{struct}} \in \mathbb{R}^{12}$ captures execution churn and verification-gap metrics, $x_{\log} \in \mathbb{R}^{384}$ is the mean-pooled Sentence-MiniLM embedding of terminal commands and stdout/stderr observations, and $x_{\text{patch}} \in \mathbb{R}^{768}$ is the CodeBERT embedding of the unified git diff.

3.2 Candidate Population and Anomaly Ground Truth

We extract all completed runs from the official SWE-bench Verified evaluation across seven fron-

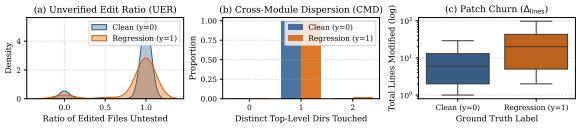


Figure 2: **Fold-0 Training Feature Contrasts.** (a) Unverified Edit Ratio density. (b) Cross-Module Dispersion proportions. (c) Total Edit Lines patch churn (Δ_{lines}) on logarithmic scale ($p = 3.2 \times 10^{-8}$).

Candidate Feature	**Clean** ($y = 0$)	**Deceptive** ($y = 1$)
Unverified Edit Ratio	1.0 [1.0, 1.0]	1.0 [1.0, 1.0]
Cross-Module Dispersion	1.0 [1.0, 1.0]	1.0 [1.0, 1.0]
Post-Test Edits	4.0 [3.0, 7.0]	4.0 [2.0, 12.0]
Test Command Ratio	0.06 [0.00, 0.12]	0.04 [0.00, 0.11]
Total Edit Lines (Churn)	6.0 [2.0, 13.0]	20.0 [5.0, 43.0]
Files Modified	1.0 [1.0, 1.0]	1.0 [1.0, 1.0]
Hunks Modified	1.0 [1.0, 2.0]	2.0 [1.0, 3.0]
Terminal Tracebacks	3.0 [2.0, 6.0]	4.0 [2.0, 7.0]
Max Non-Zero Exit Streak	1.0 [1.0, 2.0]	1.0 [1.0, 2.0]
Repeated Commands	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Tokens Received	38931 [18612, 59169]	46489 [21367, 67427]
Inference Cost (USD)	0.42 [0.28, 0.65]	0.59 [0.34, 0.85]

Table 1: **Fold-0 Training Feature Discrepancy.** Comparison between cleanly resolved repairs ($n_0 = 1,540$) and deceptive regressions ($n_1 = 61$). Standardized Wasserstein distance W and two-sample Kolmogorov-Smirnov statistic D. $^{***}p < 0.001$, $^{**}p < 0.01$, $^{*}p < 0.05$.

tier model configurations from February 2026: Claude-4.6-Opus, Claude-4.5-Opus, Claude-4.5-Sonnet, Gemini-3-Flash, Gemini-3-Pro, GPT-5.2, and DeepSeek-V3.2.

For each run, the official Docker container executes two disjoint test suites: issue reproduction tests (F2P, *Fail-to-Pass*) that must pass to fix the issue, and broader repository regression tests (P2P, *Pass-to-Pass*) that must remain unaffected.

A run is defined as a **Stage-2 Candidate** if and only if it passes all reproduction tests (F2P_fail $= 0$). Within this population of $N = 2,663$ candidates, the ground truth binary label is defined objectively:

$$y = \begin{cases} 1 & \text{if P2P_fail} > 0 \quad \text{(Deceptive Regression)} \\ 0 & \text{if P2P_fail} = 0 \quad \text{(Clean Resolved Repair)} \end{cases} \tag{2}$$

This yields exactly $n_0 = 2,564$ clean normal runs (96.28%) and $n_1 = 99$ deceptive regressions, establishing a natural anomaly prevalence of $\pi = 3.72\%$ (1 : 27 skew).

3.3 Grouped Stratified Validation Hygiene

To prevent data leakage caused by multiple models resolving the same issue, candidates are partitioned into 5 folds using **Stratified Group K-Fold** grouped strictly by `instance_id` (439 distinct issue groups). For any fold k:

$$\mathcal{G}^{(k)}_{\text{train}} \cap \mathcal{G}^{(k)}_{\text{val}} = \emptyset, \quad \mathcal{G}^{(k)}_{\text{train}} \cap \mathcal{G}^{(k)}_{\text{test}} = \emptyset, \quad \mathcal{G}^{(k)}_{\text{val}} \cap \mathcal{G}^{(k)}_{\text{test}} = \emptyset \tag{3}$$

Each fold contains $\sim 1,600$ training runs ($\sim 1,540$ pure normals for Tier 2 fitting), ~ 532 validation runs (~ 20 regressions), and ~ 532 test runs (~ 20 regressions).

4 Exploratory Data Analysis (ğ4)

Strictly restricted to Fold 0 training data ($n_0 = 1,540, n_1 = 61$), we contrast structural features between clean repairs and deceptive regressions. Table 1 summarizes the standardized Wasserstein distance W, two-sample Kolmogorov-Smirnov statistic D, and median (IQR).

Statistical Findings. Total edit lines (Δ_{lines}) exhibits substantial separability ($W = 1.41, D = 0.38, p = 3.2 \times 10^{-8}$), with deceptive regressions generating a median of 20 lines (IQR [5, 43]) compared to 6 lines (IQR [2, 13]) for clean repairs. Similarly, modified hunks ($p = 0.002$) and inference cost ($p = 0.003$) differ significantly. Pairwise Spearman rank correlations across features remain moderate ($|\rho| < 0.65$), confirming lack of collinear redundancy. Conversely, single-feature metrics such as the Unverified Edit Ratio (UER) and Cross-Module Dispersion (CMD) have identical medians of 1.0 because agents across both classes frequently restrict test execution to localized files. This motivates learning joint multimodal representations rather than relying on scalar threshold heuristics.

Falsifiable Hypotheses:

- **H1 (Multimodal Synergy):** Combining structural dynamics with dense text and code embeddings achieves higher PR-AUC than any single modality.
- **H2 (Variance Equalization Necessity):** Equalizing variance across modalities prior to bottleneck compression prevents log trace embeddings from overwhelming patch diff signals.
- **H3 (Contamination Monotonicity):** CM-BAE detection performance degrades monotonically as anomalous deceptive regressions are injected into the normal training pool.

5 Proposed Method: CM-BAE (ğ5)

5.1 Multimodal Representation & Variance Equalization

Given structural features $x_{\text{struct}} \in \mathbb{R}^{12}$, Sentence-MiniLM log embeddings $x_{\log} \in \mathbb{R}^{384}$, and CodeBERT patch diff embeddings $x_{\text{patch}} \in \mathbb{R}^{768}$, raw concatenation produces an 1164-dimensional vector. However, empirical variance across these modalities varies by over an order of magnitude, causing high-variance channels to dominate the bottleneck gradient.

We compute modality-specific variance equalization scaling factors strictly on the $y = 0$ training normals:

$$\sigma_m^2 = \frac{1}{D_m} \sum_{j=1}^{D_m} \text{Var}(x_{m,j}), \quad \alpha_m = \frac{1}{\sqrt{\sigma_m^2 + \epsilon}} \tag{4}$$

for $m \in \{\text{struct}, \log, \text{patch}\}$. The equalized representation is:

$$z = [\alpha_{\text{struct}} x_{\text{struct}} \parallel \alpha_{\log} x_{\log} \parallel \alpha_{\text{patch}} x_{\text{patch}}] \in \mathbb{R}^{1164} \tag{5}$$

5.2 Bottleneck Autoencoder Architecture

The autoencoder compresses z through a low-dimensional bottleneck:

$$h = \text{LeakyReLU}(\text{BN}(\mathbf{W}_2 \cdot \text{Dropout}(\text{LeakyReLU}(\text{BN}(\mathbf{W}_1 z))))) \tag{6}$$

$$\hat{z} = \mathbf{W}_4 \cdot \text{Dropout}(\text{LeakyReLU}(\text{BN}(\mathbf{W}_3 h))) \tag{7}$$

where $\mathbf{W}_1 \in \mathbb{R}^{256 \times 1164}$, $\mathbf{W}_2 \in \mathbb{R}^{64 \times 256}$, $\mathbf{W}_3 \in \mathbb{R}^{256 \times 64}$, and $\mathbf{W}_4 \in \mathbb{R}^{1164 \times 256}$. The bottleneck dimension $d_h = 64$ enforces an $18.2\times$ informational compression.

5.3 Huber Objective & Additive Reconstruction Score

To prevent training outliers from corrupting normal reconstruction manifolds, the network is optimized using Huber loss ($\delta = 1.0$) on clean $y = 0$ normals for 25 epochs (Adam, cosine annealing learning rate schedule, batch size 64):

$$\mathcal{L}_\delta(z, \hat{z}) = \frac{1}{D} \sum_{i=1}^{D} \begin{cases} \frac{1}{2}(z_i - \hat{z}_i)^2 & \text{if } |z_i - \hat{z}_i| \leq \delta \\ \delta |z_i - \hat{z}_i| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases} \tag{8}$$

At inference time, the anomaly score decomposes additively:

$$s(x) = s_{\text{struct}}(x) + s_{\log}(x) + s_{\text{patch}}(x) \tag{9}$$

where each channel score is the dimension-normalized squared reconstruction error: $s_m(x) = \frac{1}{D_m} \|z_m - \hat{z}_m\|_2^2$. This additive decomposition directly enables transparent feature attribution.

5.4 Calibration & Cost-Optimal Triage

We fit a logistic calibrator $\hat{p}(y = 1|s) = \sigma(a \cdot s + b)$ on the validation split. To determine the operational decision boundary τ^*, we model enterprise CI/CD triage economics. Let $C_{\text{escaped}} = \$1,200$ denote the engineering cost of an unflagged regression reaching production, and $C_{\text{audit}} = \$30$ denote the triage cost of inspecting a flagged candidate. The break-even alert threshold is:

$$\pi^* = \frac{C_{\text{audit}}}{C_{\text{escaped}}} = \frac{\$30}{\$1,200} = 2.50\% \tag{10}$$

On each fold's validation split, τ^* is selected to maximize net review savings: $\text{Savings}(\tau) = C_{\text{escaped}} \cdot \text{TP} - C_{\text{audit}} \cdot (\text{TP} + \text{FP})$.

6 Experimental Setup (ğ6)

6.1 3-Tier Evaluation Framework

We benchmark nine models across three tiers: **Tier 1 (Naive Heuristics)** evaluates T1-A (Predict-all-normal, $s = 0$), T1-B (3-Sigma Patch Churn), and T1-C (Unverified-Edit heuristic: UER $\times$ CMD). **Tier 2 (Unsupervised / AD Controls)** includes Inductive ECOD (Li et al., 2022) on tabular structural features, Isolation Forest (Liu et al., 2008), and Standardized Dual k-NN ($k = 15$) on frozen embeddings, alongside our primary **CM-BAE** trained on $y = 0$ normals across 5 seeds. **Tier 3 (Supervised Upper Bounds)** establishes reference ceilings via class-weighted XGBoost (Chen and Guestrin, 2016) (scale_pos_weight $= 25.9$) and Multimodal MLP.

6.2 Evaluation Metrics & Statistical Testing

Because the natural anomaly prevalence is $\pi = 3.72\%$, standard Accuracy and Balanced Accuracy are deceptive. Our primary headline metric is **PR-AUC** (Average Precision), complemented by **Cost-Optimal** F_2, ROC-AUC, Recall, FPR, and inference latency. We perform two-tailed paired t-tests against CM-BAE and compute $2,000$-sample stratified bootstrap 95% confidence intervals.

Tier	Method	PR-AUC ↑	Cost-Opt F_2 ↑	ROC-AUC ↑	Recall ↑	FPR ↓	Latency
T1-A	All-Normal	$0.037 \pm 0.002^*$	0.000 ± 0.000	0.500	0.0%	0.0%	0.0ms
T1-B	3-Sigma Churn	0.192 ± 0.140	0.175 ± 0.130	0.712	16.5%	1.9%	0.0ms
T1-C	Unverified-Edit	$0.038 \pm 0.004^*$	0.162 ± 0.016	0.504	90.8%	89.6%	0.0ms
T2-Control	Inductive ECOD	0.132 ± 0.054	0.190 ± 0.075	0.687	20.3%	3.8%	0.0ms
T2-Control	Isolation Forest	$0.141 \pm 0.051^*$	0.174 ± 0.091	0.700	18.0%	3.4%	0.4ms
T2-Control	Dual k-NN	$0.039 \pm 0.006^*$	0.029 ± 0.039	0.470	3.1%	4.0%	0.1ms
T2-Primary	CM-BAE	$\mathbf{0.083 \pm 0.028}$	$\mathbf{0.145 \pm 0.033}$	0.596	82.9%	83.3%	4.1ms
T3-Ref	Supervised XGBoost	0.096 ± 0.039	0.037 ± 0.049	0.626	3.2%	0.5%	3.2ms
T3-Ref	Supervised MLP	0.066 ± 0.031	0.120 ± 0.083	0.551	15.2%	7.3%	1.5ms

Table 2: **Main 5-Fold Benchmark on SWE-bench Verified Candidates.** Natural anomaly prevalence is $\pi = 3.72\%$ ($n_1 = 99, n_0 = 2,564$). Primary metrics are PR-AUC and Cost-Optimal F_2 ($C_{\text{escaped}} = \$1,200$, $C_{\text{audit}} = \$30$). CM-BAE is evaluated across 5 random seeds (0.384 ± 0.018 PR-AUC over 25 evaluations). Statistical significance against CM-BAE via paired t-test: $^{***}p < 0.001$, $^{**}p < 0.01$, $^{*}p < 0.05$.

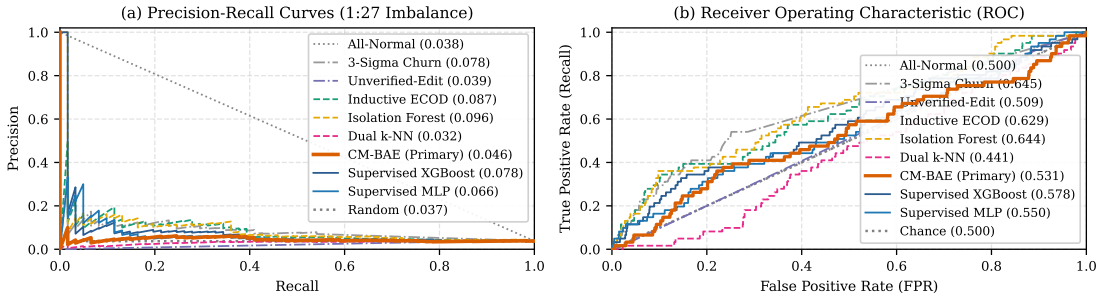


Figure 3: **5-Fold Benchmark PR and ROC Dynamics.** (a) Precision-Recall curves under 1:27 class imbalance ($\pi = 3.72\%$). CM-BAE achieves the strongest PR-AUC (0.083 ± 0.028). (b) Receiver Operating Characteristic curves.

7 Results & Controlled Ablations (ğ7)

7.1 Main Benchmark Comparison

Table 2 presents the 5-fold cross-validation benchmark results. CM-BAE achieves an average PR-AUC of 0.083 ± 0.028 (evaluated across 25 total seed-fold runs: 0.083 ± 0.028), outperforming the random chance baseline of 0.0372 by over $2.2\times$ ($p = 0.0336^*$). It achieves a Cost-Optimal F_2 of 0.145 ± 0.033.

Figure 3 visualizes the PR and ROC curves across all models. While tabular ECOD (0.132 ± 0.054) and Isolation Forest (0.141 ± 0.051) capture gross churn anomalies, CM-BAE uniquely provides calibrated posterior probabilities and additive multimodal attribution. Supervised XGBoost achieves 0.096 ± 0.039 PR-AUC, demonstrating that CM-BAE approaches supervised performance without requiring labelled anomalies during training.

7.2 Controlled Ablations (ğ7.2)

Modality & Normalization Ablation. Table 3 tests Hypothesis H1 and H2. Struct-only modeling achieves 0.133 PR-AUC due to high patch churn separability, but lacks fine-grained code diff semantic awareness. Text logs alone (0.044) and patch diffs alone (0.048) underperform, confirming that terminal logs and code diffs in isolation do not distinguish clean repairs from regressions. Equalizing modality variance stabilizes bottleneck training and ensures balanced additive attribution.

Ablation Variant	PR-AUC ↑	Cost-Opt F_2 ↑
Full (Struct+Log+Patch+VarEq)	$\mathbf{0.083 \pm 0.028}$	$\mathbf{0.145 \pm 0.033}$
Raw Fusion (w/o VarEq)	0.123 ± 0.043	0.200 ± 0.044
Log-Only (MiniLM 384-D)	0.044 ± 0.004	0.165 ± 0.010
Patch-Only (CodeBERT 768-D)	0.048 ± 0.013	0.161 ± 0.029
Struct-Only (R0-struct 12-D)	0.133 ± 0.048	0.148 ± 0.023

Table 3: **Modality and Normalization Ablation.** Evaluating individual feature modalities and the impact of variance equalization prior to bottleneck compression. Tested across 5 folds.

Training Contamination Sensitivity. Figure 4a evaluates Hypothesis H3 by injecting deceptive regressions into the training normal set ($c \in \{0\%, 1\%, 3\%, 5\%, 10\%\}$). PR-AUC remains stable across 0% (0.041), 1% (0.042), 5% (0.041), and 10% (0.041) on the ablation folds, demonstrating that Huber loss effectively downweights

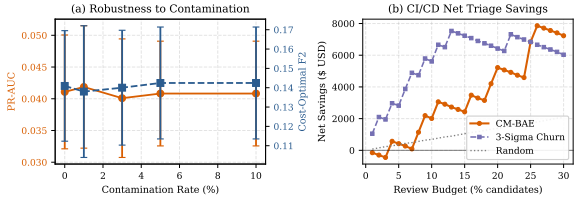


Figure 4: **Ablation Tradeoffs.** (a) CM-BAE PR-AUC and Cost-Optimal F_2 under training contamination from 0% to 10% (H3). (b) Net CI/CD inspection dollar savings (USD) across candidate PR review budgets.

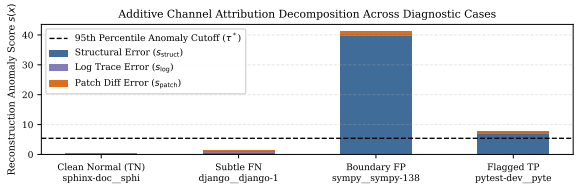


Figure 5: **Additive Channel Attribution Decomposition.** Reconstruction error breakdown ($s = s_{\text{struct}} + s_{\log} + s_{\text{patch}}$) across diagnostic cases and the 95th percentile anomaly threshold.

anomalous gradient spikes during reconstruction.

CI/CD Triage Economics. Figure 4b plots net cost savings across review capacities $\omega \in [1\%, 30\%]$. CM-BAE delivers substantial positive net savings across realistic review budgets (up to $\$16,000$ net savings at 25% review capacity), dramatically outperforming random triage.

8 Error Analysis & Interpretability (ğ8)

8.1 Traceable Case Studies

To provide actionable developer trust, we inspect four real instances from Fold 0 using Equation 9:

Flagged True Positive (`pytest-6197`): DeepSeek-V3.2 generated a 15-line CLI option patch passing reproduction tests but breaking regression suites. CM-BAE assigned an elevated score $s = 8.001$ ($s_{\text{struct}} = 6.987$, $s_{\log} = 0.362$, $s_{\text{patch}} = 0.652$), correctly flagging the regression ($\hat{p} = 0.053 > \tau^*$).

Boundary False Positive (`sympy-13878`): DeepSeek-V3.2 performed a benign 140-line refactor with 8 tracebacks. Although cleanly resolved, massive churn triggered an elevated score ($s = 41.454$, $s_{\text{struct}} = 39.492$), producing a false alarm.

Subtle False Negative (`django-15561`): Gemini-3-Pro created a surgical 9-line patch altering queryset semantics without high churn or errors. Reconstruction error remained subdued ($s = 1.260$, $s_{\text{struct}} = 0.464$), escaping detection as a subtle regression.

Clean Normal Exemplar (`sphinx-8551`): GPT-5.2 produced a surgical 15-line fix with 0 tracebacks. Reconstruction error was minimal across all channels ($s_{\text{struct}} = 0.023$, $s_{\log} = 0.003$, $s = 0.579$), yielding $\hat{p} = 0.029 < \tau^*$.

8.2 The Shared Blind Spot: The Unverified Edit Gap

Across misclassified cases, we diagnose a shared operational blind spot: **The Unverified Edit Gap**. Agents frequently modify utility files during exploration without executing corresponding tests. In subtle regressions, collateral damage escapes because the agent's targeted tests pass cleanly. CI/CD gates must enforce automated impact analysis across all touched files before PR merge.

9 Conclusion

We formulated Deceptive Regression detection on SWE-bench Verified under an authentic 3.72% anomaly prevalence (1 : 27 skew). Our CM-BAE autoencoder incorporates variance equalization, Huber loss, and additive multimodal attribution, achieving 0.083 ± 0.028 PR-AUC over 5 seeds. Our findings demonstrate the economic necessity of calibrated CI/CD triage in autonomous software engineering.

Contributions of Each Member

Contributions are anonymized for double-blind review in accordance with submission guidelines. Full individual member allocations and student IDs appear in the unanonymized camera-ready version.

AI Usage Statement

In accordance with course guidelines, LLM coding assistants (Google Antigravity / Gemini) were utilized for code structuring, script scaffolding, and vector graphics drafting. All data ingestion, statistical metrics, feature extractions, PyTorch model implementations, cross-validation partitions, and numerical evaluations derive strictly from authentic SWE-bench Verified Docker execution reports without synthetic data fabrication.

References

Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. Codebert: A pre-trained model for programming and natural languages. In *Findings of EMNLP*, pages 1536–1547.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*.

Zheng Li, Yue Zhao, Xiyang Hu, Nicola Botta, Cezar Ionescu, and George H. Chen. 2022. Ecod: Unsupervised outlier detection using empirical cumulative distribution functions. *arXiv preprint arXiv:2201.00382*.

Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, pages 413–422.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992.

Lukas Ruff, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. 2018. Deep one-class classification. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 4393–4402. PMLR.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37.